DevOps Fundamentals Assignment 2

E-Commerce Peak Season Readiness

Preparing an online retailer for tenfold festival-sale traffic

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course / Subject	Java Full Stack
Instructor	Mr. Pravin
Submission date	October, 04, 2026

Table of Contents

Table of Contents 2
1. Executive Summary 3
2. Business Scenario and Objectives 3
  2.1 Scenario 3
  2.2 Objectives 3
3. Stakeholders 3
4. Current Challenges and Risks 4
  4.1 Challenges and Risks 4
  4.2 Bottleneck Analysis 4
5. Assumptions and Constraints 5
  5.1 Given Requirements (from the assignment) 5
  5.2 Assumptions (made by the author) 6
  5.3 Constraints 6
6. Proposed Solution (Step-by-Step Approach) 6
  6.1 Readiness Approach and Principles 7
  6.2 Identify Bottlenecks and Plan Capacity 7
  6.3 Deployment Strategy 8
  6.4 Monitoring Requirements 9
  6.5 Incident Response Process 10
  6.6 Rollback Mechanisms 11
  6.7 Align Business, Development, Testing and Operations 13
7. Implementation Plan 13
  7.1 Eight-Week Countdown 13
  7.2 Go/No-Go Checklist 14
8. Measurable Success Criteria 14
9. Recommendations and Lessons Learned 15
  9.1 Recommendations 15
  9.2 Lessons Learned 15

1. Executive Summary

Problem. An online retailer expects ten times its normal traffic during a festival sale. Operations teams fear downtime, while developers keep shipping new features. These two goals pull in opposite directions, and a single outage during the sale could cost a large share of the year's revenue.

Solution. An eight-week readiness plan with six parts: find the bottlenecks by load testing, scale and protect the critical checkout path, control changes through a tiered freeze (features keep being built but are released safely), monitor business and technical signals, run a clear incident-response process, and prepare tested rollbacks. One shared plan and calendar align business, development, testing and operations.

Expected result. The sale runs without major downtime; if something does break, it is detected within minutes and fixed or rolled back quickly. Section 8 defines how success will be measured.

2. Business Scenario and Objectives

2.1 Scenario

The retailer's festival sale is a fixed, public event. Customers arrive in waves, often triggered by marketing messages, and they expect the site to work. Operations teams worry that any instability will cause downtime. Developers, meanwhile, are still delivering features the business wants. Every release before the sale is a possible source of failure.

Simple explanation: peak readiness means preparing the system, the process and the people so that high demand does not become an outage.

2.2 Objectives

#	Objective	Why it matters
O1	Keep the site available during the sale	Every minute of downtime is lost sales and lost trust
O2	Protect the critical path: browse, cart, checkout, payment	This is where revenue is made
O3	Keep delivering features without adding risk	The business still needs progress
O4	Detect problems within minutes	Fast detection limits the damage
O5	Recover quickly with clear roles and tested rollbacks	Mistakes will happen; recovery speed decides the impact
O6	Align business, development, testing and operations	Shared goals prevent surprises and blame

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, the sale's success.

Stakeholder	Role	What they need
Business leadership	Own revenue and risk decisions	Sales targets met; clear Go/No-Go information
Marketing	Plan campaigns and notifications	A site that survives their campaigns; warning of traffic spikes
Product owners	Prioritise features	Important features delivered without risking the sale
Developers	Build and fix the software	Clear change rules; fast feedback; safe release methods

Stakeholder	Role	What they need
QA / Testers	Check quality and performance	Time and environments for load and regression testing
Operations / SRE	Run and monitor production	Stability, capacity, monitoring, runbooks
Security team	Protect customers and data	Defence against bots, fraud and attacks during the spike
Customer support	Answer customer issues	Early warning of problems; status updates to share
Partners (payment, delivery, SMS)	Provide external services	Advance notice of expected volume
Finance	Track cost and revenue	Predictable cost of extra capacity
Customers	Shop during the sale	A fast, reliable, secure shopping experience

4. Current Challenges and Risks

4.1 Challenges and Risks

#	Challenge	Impact	Risk
C1	Ten times the usual traffic in a short time	Systems that work today may fail under load	High
C2	Operations and developers have conflicting goals (stability versus new features)	Late risky releases; blame when something breaks	High
C3	Bottlenecks are not known or measured	The weakest part fails first, unexpectedly	High
C4	Changes close to the event are a common cause of outages	Self-inflicted downtime	High
C5	Monitoring may show servers but not business results	Slow detection of real customer pain	High
C6	No agreed incident-response process or roles	Confusion and delay during an outage	High
C7	Rollback not planned or tested	Long recovery; panic fixes	High
C8	Dependence on third parties (payment, delivery, SMS)	Their limits become the site's limits	High
C9	Teams plan separately (marketing, product, tech)	Campaign spikes arrive without warning	Medium
C10	Bots, fraud and attacks increase during sales	Capacity wasted; security incidents	Medium
C11	Staff fatigue during long sale days	Mistakes in judgement	Medium

Risk level is a judgement based on the likely effect on revenue, customers and reputation. It is not measured data.

4.2 Bottleneck Analysis

Simple explanation: a bottleneck is the slowest or smallest part of a system. It decides how much traffic the whole system can handle, like the narrowest part of a bottle. Making other parts faster does nothing until the bottleneck is fixed, and fixing one often reveals the next.

The table lists the **most likely bottlenecks** for a typical online retailer. They are hypotheses to be confirmed by testing, not facts about a specific system (Assumption A1).

Layer	Likely bottleneck at ten times traffic	How to confirm
Web and application servers	CPU and connection limits; slow scaling-up	Load test; watch CPU, memory and response time
Database	Connection limits, slow queries, locks on orders and stock	Slow query logs; connection counts; load test
Cache	Cold cache at the start of the sale; many requests hitting the database at once	Cache hit ratio during tests
Search and catalogue	Heavy read traffic on popular products	Load test with realistic search patterns
Inventory service	Many buyers of the same item; overselling from race conditions	Concurrent purchase tests on a hot item
Payment gateway (third party)	Rate limits and slower responses under load	Ask the provider; test in their sandbox; track timeouts
Images and static content	Origin servers overloaded without a CDN	Check CDN hit ratio and origin traffic
Login and sessions	Spike of logins at sale start	Load test the login flow
Background jobs	Order emails and notifications pile up	Watch queue length and processing time
Deployment process	Slow, manual or risky releases	Time a release; review last incidents
People and process	No clear owner, unclear decisions, no on-call plan	Readiness review; run a practice incident

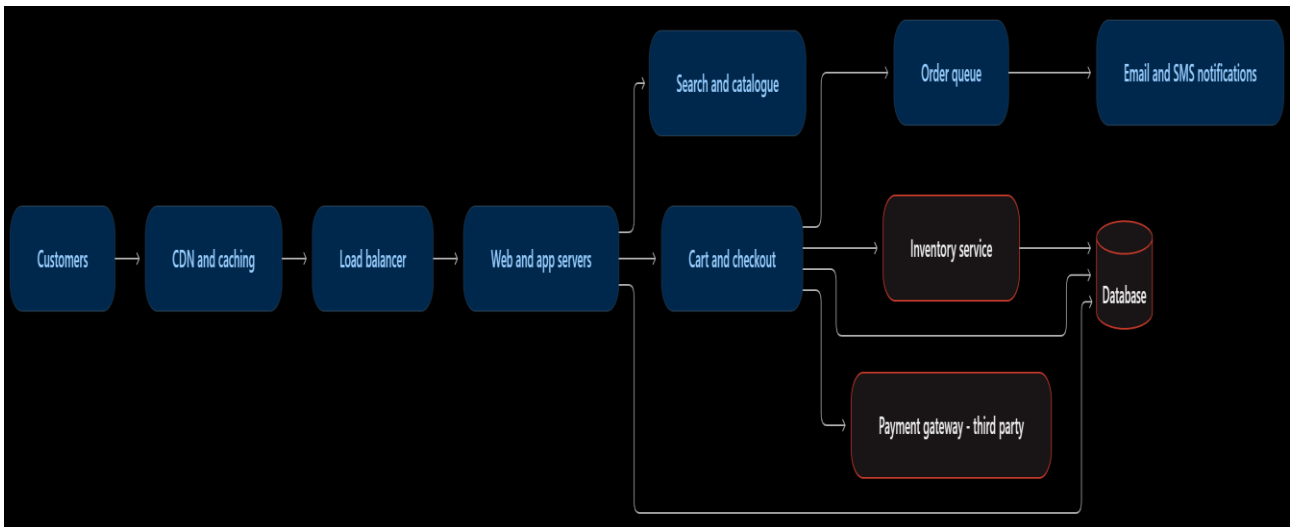


DIAGRAM 1: CUSTOMER REQUEST PATH AND BOTTLENECKS

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**, so the two are never confused.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	An online retailer expects traffic to increase tenfold during a festival sale
R2	Operations teams are concerned about downtime
R3	Developers continue delivering features
R4	Identify bottlenecks

#	Requirement
R5	Create a deployment strategy
R6	Define monitoring requirements
R7	Establish incident-response processes
R8	Plan rollback mechanisms
R9	Align business, development, testing and operations teams
R10	Deliver a report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	The assignment gives no system details, so bottlenecks, numbers, thresholds and examples in this report are **illustrative**	Real values come from measuring the real system
A2	"Tenfold" means ten times normal peak traffic, and the forecast could be wrong; the plan tests above it	If real traffic is higher, the degradation plan (Section 6.2) applies
A3	The sale date is fixed and about eight weeks away	A shorter time means fewer improvements; priorities would shrink to the checkout path
A4	The site runs on cloud infrastructure that can scale up on demand	On fixed hardware, capacity must be bought earlier
A5	A basic CI/CD pipeline already exists	Pipeline setup would be added to the early weeks
A6	Payments, delivery and SMS use external providers	Fewer third-party risks if built in-house
A7	The sale lasts three to five days, with spikes at announced times	Staffing and on-call plans would change
A8	Tool names (for example Kubernetes, k6, Prometheus, Grafana) are examples only	Any equivalent tools can be used
A9	No baseline data is given, so targets in Section 8 are illustrative	Re-base targets on real measurements

5.3 Constraints

- **Fixed deadline:** the sale date cannot move.
- **Cost:** extra capacity costs money, and over-provisioning has a limit.
- **Business needs features:** a complete stop to development is not acceptable, so change must be controlled, not banned.
- **Third parties:** payment and delivery providers have their own limits outside the retailer's control.
- **Data accuracy:** stock levels and payments must stay correct under load.
- **Security and payment rules:** customer and card data must stay protected.
- **People:** limited staff and the risk of fatigue during the event.
- **Scope:** this is a design and planning exercise; no real systems are changed.

6. Proposed Solution (Step-by-Step Approach)

The solution has seven parts. Sections 6.2 to 6.7 map directly to the assignment's tasks: bottlenecks, deployment strategy, monitoring, incident response, rollback and team alignment.

6.1 Readiness Approach and Principles

Core idea: protect the money path first. The critical path is **browse, search, cart, checkout, payment, confirmation**. When resources run short, less important features are switched off so the critical path stays up.

Tier	Services (illustrative)	Under heavy load
Tier 1: critical	Checkout, payment, inventory, login, order confirmation	Always protected; extra capacity; no risky changes
Tier 2: important	Search, product catalogue, cart	Scaled up; simplified if needed
Tier 3: nice to have	Recommendations, reviews, personalisation, wishlists	Switched off first to free resources

Five principles: prepare early, protect the critical path, observe everything that matters, respond with clear roles, and recover fast (restore service first, find the root cause later).

6.2 Identify Bottlenecks and Plan Capacity

The method is: **measure, find the weakest point, fix it, test again**, repeated until the target load is handled with headroom.

Types of load testing

Test	Question it answers	Example
Baseline	How does the system behave on a normal day?	Normal traffic for 30 minutes
Load	Can it handle the expected sale traffic?	Ramp to the expected tenfold peak
Stress	Where does it break and how does it fail?	Push to 1.5 to 2 times expected peak
Spike	Can it survive a sudden jump, such as a push notification?	Jump from normal to peak in one minute
Soak	Does it stay healthy for hours?	Hold high load for 4 to 8 hours

Rule: test above the forecast (for example 1.5 times expected peak), because forecasts are often wrong.

```
// k6 load test example (illustrative); targets are virtual users
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '5m',  target: 500  },   // normal peak
    { duration: '10m', target: 5000 },   // expected sale peak
    { duration: '10m', target: 7500 },   // stress: 1.5x expected
    { duration: '5m',  target: 0    },
  ],
  thresholds: {
    http_req_duration: ['p(95)<2000'],   // 95% under 2 seconds
    http_req_failed:   ['rate<0.01'],    // under 1% errors
  },
};

export default function () {
  const res = http.get('https://test.example-shop.com/checkout');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Capacity and protection techniques

Technique	Problem it solves	Note
Pre-scaling before known spikes	Autoscaling reacts too slowly to sudden jumps	Scale up before the sale and before announced campaigns
Autoscaling	Unexpected growth during the sale	Set sensible limits; test that it works
CDN and caching	Repeated requests for the same content	Warm the cache before the sale
Database tuning	Slow queries; too many connections	Add indexes; connection pooling; read replicas
Queues for background work	Bursts of emails and notifications	Process orders first, notifications later
Rate limiting and bot protection	Bots and abusive traffic wasting capacity	Protect login and checkout
Virtual waiting room	More visitors than the checkout can serve	Lets customers in at a safe rate instead of crashing
Graceful degradation	Overload beyond forecast	Turn off Tier 3 features by feature flag
Static fallback pages	Total failure of a component	Friendly message instead of an error
Partner capacity check	Third-party limits	Confirm volume limits with payment and delivery providers

6.3 Deployment Strategy

The tension: developers want to ship; operations want stability. **The resolution:** do not choose one. Separate **deploying** code from **releasing** features, and tighten change rules gradually as the sale approaches.

Simple explanation: with a feature flag, developers can put new code into production switched off (dark), and the business turns it on later when it is safe. Development keeps moving, but customers see no change.

Tiered change policy

Period	Allowed changes	Approval	Release technique
Normal (until 4 weeks before)	All planned changes	Normal review	Canary or blue-green; feature flags for risky features
Stabilise (4 weeks to 1 week before)	Low-risk changes; bug fixes; no database structure changes in the last 2 weeks	Test evidence required	Canary; new features stay behind flags
Freeze (last week through the sale)	Tier 1 systems frozen; only critical bug and security fixes	Emergency approval by the Incident Commander and business owner	Small, tested, reversible changes only
Sale days	Emergency fixes and configuration toggles only	War-room decision	Feature flags, scaling changes, rollbacks
Thaw (after the sale)	Resume gradually; ship the features held back	Normal review, starting with small changes	Normal methods

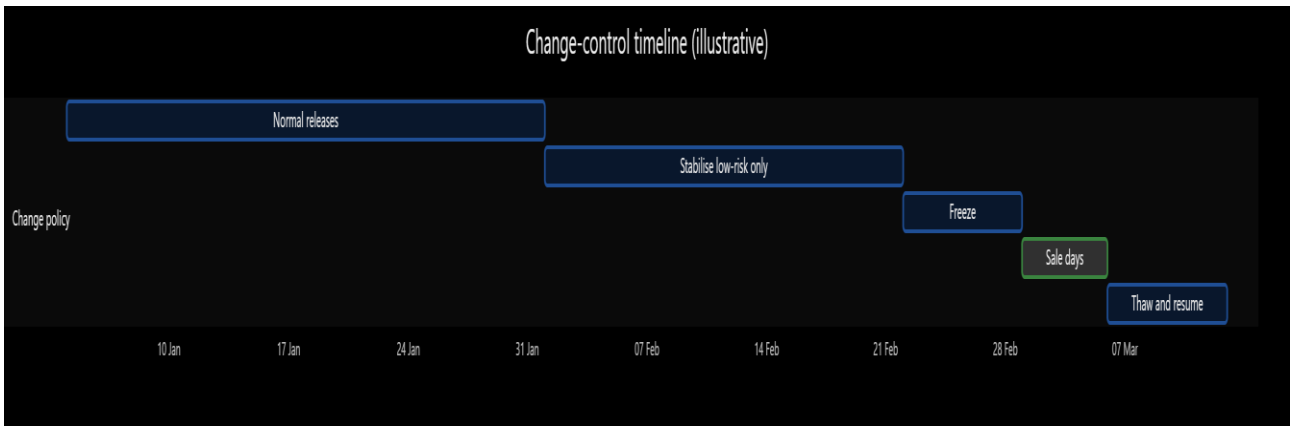


DIAGRAM 2: CHANGE-CONTROL TIMELINE

Safe deployment techniques

Technique	How it works	Benefit
Blue-green deployment	Two identical environments; switch traffic from old to new	No downtime; instant switch back
Canary release	Release to a small share of customers first	Problems affect few customers
Rolling update	Replace servers gradually	Capacity stays up during release
Feature flags	Turn features on and off without redeploying	Separates deployment from release; instant kill switch
Backward-compatible database changes	Add new structure first, remove the old later (expand and contract)	The old application version still works, so rollback is simple
Same artifact everywhere	Build once; promote the same package through environments	What was tested is what is released

- **Never deploy at peak hours,** even during the freeze. Emergency fixes go out in a quiet window or gradually.
- **Pre-release checklist:** tests passed, rollback plan written, monitoring ready, on-call person aware.

6.4 Monitoring Requirements

Simple explanation: monitoring means watching the system so the team knows about problems before customers complain. During a sale it must show **business results**, not just server health.

Area	Metric	Alert example (illustrative)	Severity
Business	Orders per minute	Drops more than 30% below forecast	High
Business	Checkout success rate	Below 95% for 3 minutes	High
Business	Payment success rate	Falls more than 3 points below normal	High
Application	Error rate	Above 1% for 2 minutes	High
Application	Response time (95th percentile)	Above 2 seconds for 5 minutes	Medium
Infrastructure	CPU and memory	Above 80% for 5 minutes	Medium
Infrastructure	Autoscaling events	Scaling at its limit	High
Database	Connections and slow queries	Connections above 85% of limit	High
Cache and queues	Cache hit ratio; queue length	Hit ratio falling; queue growing steadily	Medium

Area	Metric	Alert example (illustrative)	Severity
Dependencies	Payment gateway timeouts	Timeouts above 2% or provider reports issues	High
Security	Bot traffic; failed logins	Sudden jump compared with normal	Medium
Customer view	Synthetic purchase test every minute	Any failure	High

Monitoring requirements

- **One war-room dashboard** on a single screen: orders, errors, speed, payments, capacity and alerts.
- **Three data types:** metrics (numbers), logs (event records) and traces (a request's journey).
- **Real-user and synthetic monitoring:** real customers' experience plus a scripted test purchase every minute.
- **Alert on customer impact,** not noise. Every alert has an owner and a runbook link.
- **Notification path:** alerts page the on-call person; unacknowledged alerts escalate after 5 minutes.
- **Data protection:** logs must never contain card details or passwords.
- **Test the monitoring itself** before the sale by triggering a deliberate alert.

6.5 Incident Response Process

Simple explanation: an incident is any event that harms customers or revenue. A good process means people know their roles, so they spend time fixing the problem instead of arguing about who is in charge.

Severity levels

Level	Meaning (illustrative)	Acknowledge	Updates
SEV1	Checkout or payment down, or large share of customers affected	Within 5 minutes	Every 15 minutes
SEV2	Major slowdown or an important feature failing	Within 15 minutes	Every 30 minutes
SEV3	Minor issue with a workaround	Within 1 hour	As needed

Roles

Role	Responsibility
Incident Commander	Leads the response, makes decisions, decides on rollback
Technical lead	Directs investigation and fixes
Communications lead	Updates the business, customer support and the public status page
Business liaison	Advises on revenue impact and customer-facing decisions
Scribe	Records timeline and actions for the postmortem

Process

1. **Detect:** an alert fires or someone reports a problem.
2. **Triage:** is customer impact real? If yes, declare an incident and set severity.
3. **Organise:** assign the Incident Commander and open the war room.
4. **Mitigate:** restore service first. If a change was made recently, roll back or switch off the feature before investigating.
5. **Communicate:** regular updates to the business, support and customers.

6. **Resolve:** confirm recovery in monitoring, then keep watching.
7. **Learn:** a blameless postmortem within five working days, with actions and owners.

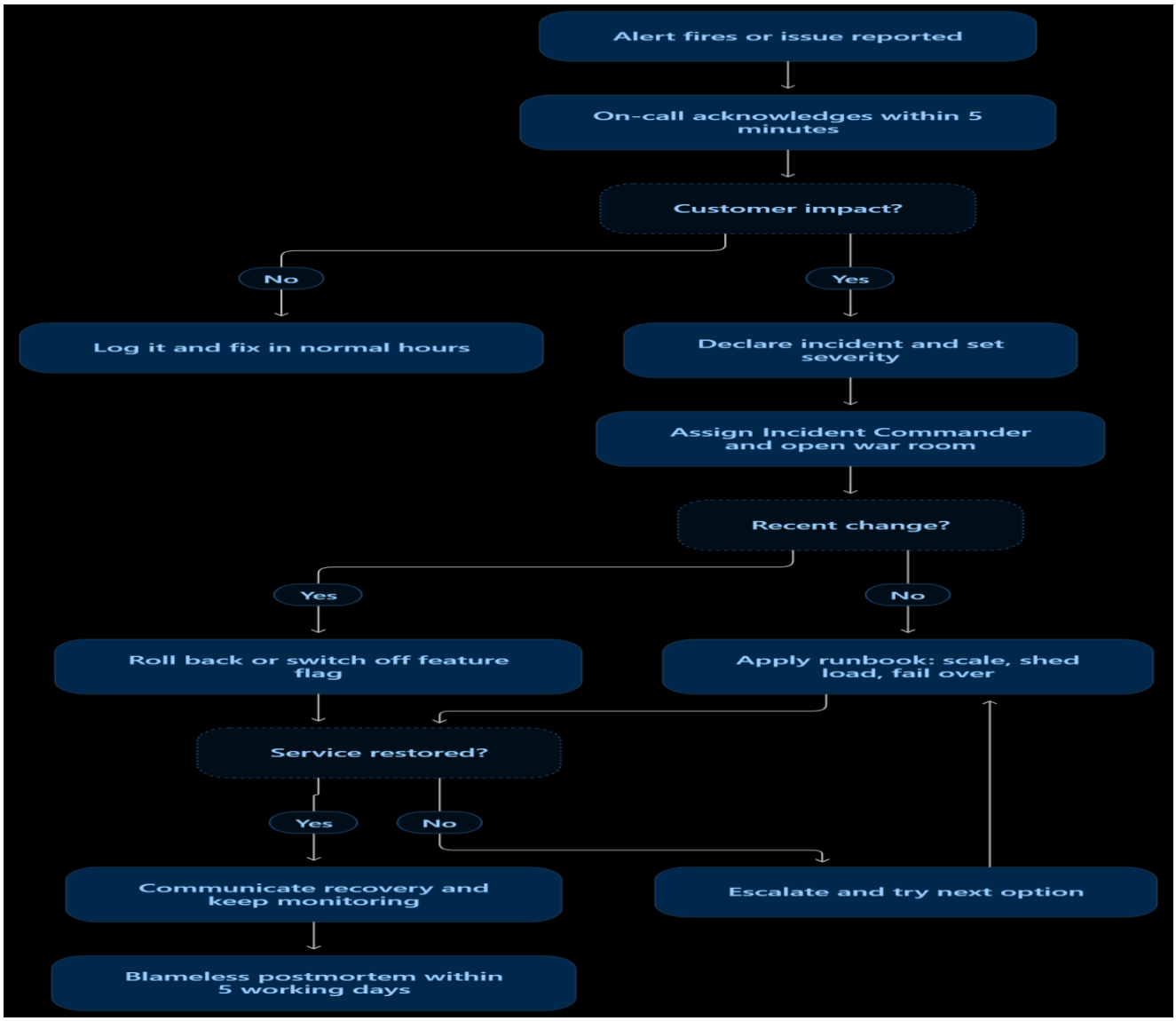


DIAGRAM 3: INCIDENT RESPONSE FLOW

Preparation

- **Runbooks** for the likely failures: payment gateway slow, database overloaded, cache failure, bot attack, stock mismatch.
- **On-call rota with shifts** so nobody works long stretches without rest.
- **War room** (physical or virtual) staffed during peak sale hours.
- **Status page and customer messages** prepared in advance.
- **Game day:** a practice run with a simulated failure before the freeze.

6.6 Rollback Mechanisms

Simple explanation: a rollback returns the system to the last known good state. The fastest safe fix in an incident is often to undo the latest change.

Situation	Mechanism	Target time
A new feature misbehaves	Feature flag kill switch (turn it off)	Seconds

Situation	Mechanism	Target time
A configuration change breaks something	Revert to the previous versioned configuration	Minutes
A bad application release	Switch traffic back (blue-green) or redeploy the previous version	Under 10 minutes
A database change causes trouble	Backward-compatible changes allow application rollback without undoing the data change	Under 10 minutes
Infrastructure change goes wrong	Revert infrastructure code and reapply	Minutes to an hour
A third-party service fails	Circuit breaker and fallback (for example, an alternative payment method)	Automatic or minutes
Traffic beyond capacity	Shed load: waiting room, rate limits, switch off Tier 3 features	Minutes
Data corruption (last resort)	Restore from backup; Incident Commander decides	Hours; avoided by prevention

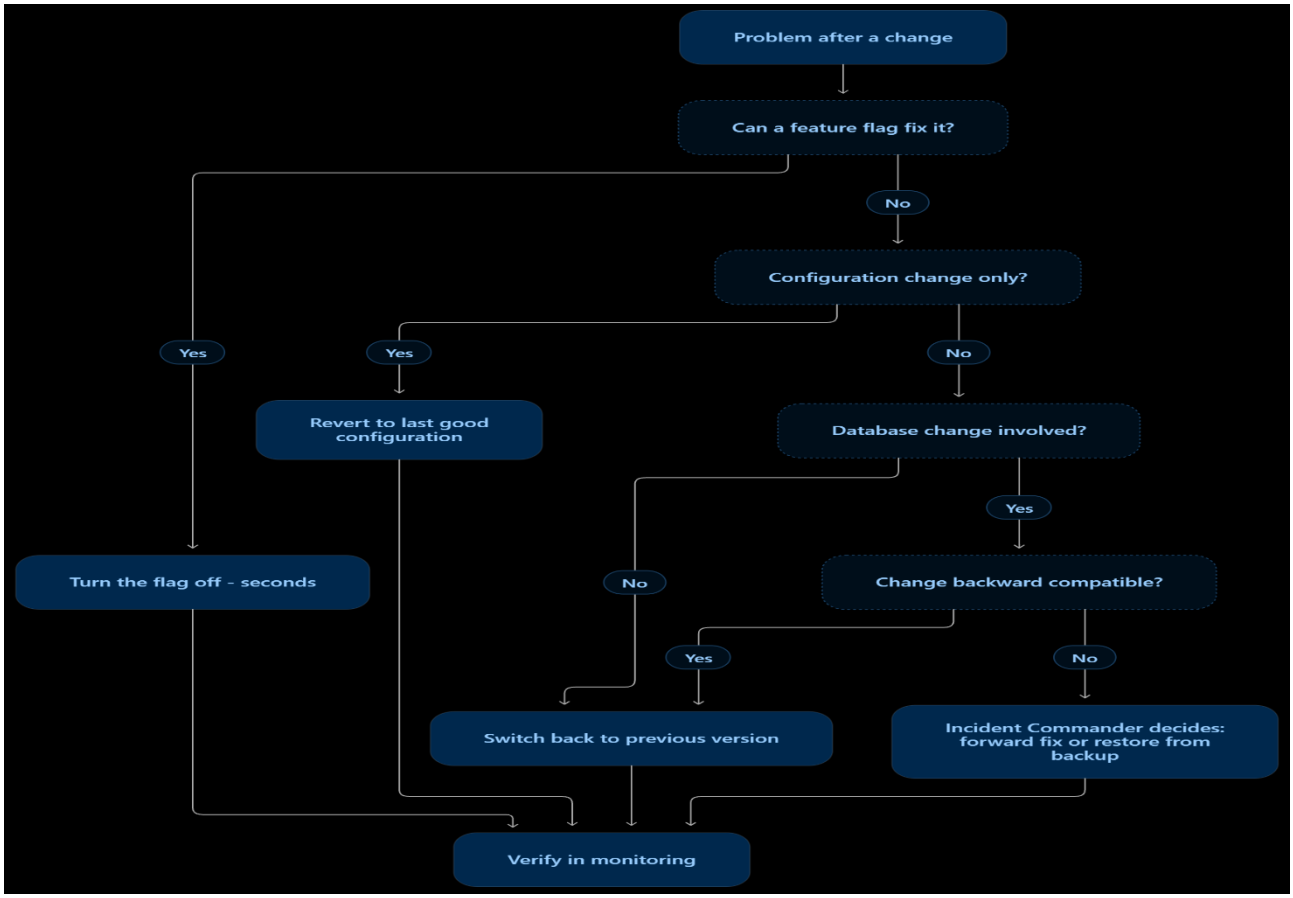


DIAGRAM 4: ROLLBACK DECISION FLOW

```
# Example commands (assuming Kubernetes and Git; tools are examples)
kubectl rollout status deployment/checkout     # watch a release
kubectl rollout undo deployment/checkout       # roll back to previous version
git revert -m 1 <merge-commit>                 # undo a merged change in code
# Feature flag kill switch (tool-specific): set 'new-recommendations' = off
```

Rollback rules

- **Rollback first, investigate later** when a SEV1 follows a recent change. The Incident Commander decides, with no debate.
- **Test every rollback** in rehearsal. A rollback that has never been tested is only a hope.

- **Design for rollback:** backward-compatible database changes, versioned configuration, flags for risky features.
- **Never rewrite shared history** to undo a change; use revert so there is a record.

6.7 Align Business, Development, Testing and Operations

Most peak-season failures come from poor coordination, not just poor technology. Alignment practices:

- **One readiness plan and one readiness lead** accountable across all teams.
- **Shared calendar** showing marketing campaigns, freeze dates and testing windows. A push notification at 8 pm creates a spike that tech must know about.
- **Weekly readiness review** from eight weeks out, reviewing risks and progress.
- **Go/No-Go meetings** one week and one day before the freeze, using a written checklist (Section 7.2).
- **Shared goals and dashboards:** everyone is judged on availability and conversion, not only their own function.
- **Product owners decide in advance** which features are must-have for the sale and which wait until after.
- **QA and Operations plan performance tests together,** using realistic customer behaviour from last year's data.

Responsibilities (RACI)

R = does the work, A = accountable, C = consulted, I = informed.

Activity	Business	Dev	QA	Ops
Forecast traffic and campaigns	A / R	C	I	C
Prioritise sale features	A / R	C	C	I
Fix bottlenecks	I	R	C	A / R
Load and regression testing	I	C	A / R	R
Change freeze and approvals	C	R	C	A / R
Monitoring and alerting	C	R	C	A / R
Incident response	C	R	C	A / R
Go/No-Go decision	A	C	R	R
Postmortem and learning	C	R	R	A / R

7. Implementation Plan

7.1 Eight-Week Countdown

The plan counts down to the sale start (week T-8 means eight weeks before).

Week	Activities	Output
T-8	Name the readiness lead; agree objectives and service tiers; measure baselines; set freeze dates; start bottleneck analysis	Readiness plan; baseline data
T-7 to T-6	Run first load tests; fix top bottlenecks; build monitoring dashboards and synthetic checks; confirm partner limits	First test report; dashboards live
T-5 to T-4	Capacity changes; caching and database tuning; autoscaling and waiting room; feature flags for risky features; second load test (stress); draft runbooks. **Stabilise period starts**	Tuned system; draft runbooks
T-3	Incident response training; rollback rehearsal; game day with a simulated failure; security review (bots, fraud)	Tested rollback; trained team

Week	Activities	Output
T-2	Low-risk changes only; final load, spike and soak tests; plan pre-scaling; finalise on-call rota and status page	Final test report
T-1	**Go/No-Go meeting; code freeze for Tier 1;** pre-scale; set up war room	Go decision; freeze in place
Sale days	War room staffed; dashboards watched; check-ins at set times; emergency-only changes	Stable sale; incident log
T+1 week	Scale back gradually; thaw freeze; postmortem and retrospective; record lessons	Postmortem report; improvement actions

7.2 Go/No-Go Checklist

#	Criterion	Owner
1	Load test passed at 1.5 times expected peak with response time and error targets met	QA and Ops
2	Top bottlenecks fixed or accepted with a mitigation	Dev and Ops
3	Rollback rehearsed and completed within target time	Ops
4	Monitoring, alerts and war-room dashboard tested	Ops
5	Runbooks written for the top failure scenarios	Ops and Dev
6	On-call rota, roles and war room confirmed	Readiness lead
7	Payment, delivery and SMS partners confirmed capacity	Business and Ops
8	Feature flags in place for all risky and Tier 3 features	Dev
9	Freeze communicated; emergency approval process agreed	Readiness lead
10	Status page and customer messages prepared	Business and Support

Rule: if a critical item (1, 3 or 4) fails, the answer is No-Go for the risky change or for the capacity plan, and the gap is fixed before proceeding.

8. Measurable Success Criteria

No baselines are given in the assignment, so all targets are illustrative (Assumption A9) and should be re-based on real data.

#	Metric	Target	How measured
S1	Checkout availability during the sale	At least 99.95%	Synthetic checks and uptime monitoring
S2	Checkout response time (95th percentile) at peak	Under 2 seconds	Application monitoring
S3	Error rate at peak	Under 1%	Application monitoring
S4	Payment success rate	Within 2 points of normal	Payment dashboard
S5	Load test result before the freeze	Passes at 1.5 times expected peak	Load test report
S6	Time to detect a SEV1 problem	Under 2 minutes	Alert timestamps
S7	Time to acknowledge / mitigate a SEV1	Under 5 minutes / under 15 minutes	Incident log
S8	Rollback time in rehearsal	Under 10 minutes	Rehearsal record
S9	Release-caused incidents during the freeze	0	Incident log

#	Metric	Target	How measured
S10	Overselling incidents	0	Order and stock reconciliation
S11	Go/No-Go checklist items complete before the freeze	100%	Checklist record
S12	Postmortems completed for SEV1 and SEV2	100% within 5 working days	Postmortem tracker
S13	Sale revenue versus forecast	At or above forecast	Business reports
S14	Staff on call for more than 12 hours in a row	0	On-call records

9. Recommendations and Lessons Learned

9.1 Recommendations

1. Start preparing about eight weeks ahead and name one readiness lead.
2. Test at realistic and higher-than-forecast load. Do not trust the forecast alone.
3. Protect the checkout path and switch off nice-to-have features first when overloaded.
4. Use feature flags so developers keep building while the business controls what customers see.
5. Control change in stages (stabilise, freeze, thaw) instead of a sudden total ban.
6. Monitor business results (orders, payments) as closely as servers.
7. Agree incident roles and rehearse both the response and the rollback.
8. Confirm capacity with third-party providers and share campaign schedules with the technical teams.

9.2 Lessons Learned

These come from designing this plan, not from a live event.

- **Changes cause as many outages as traffic does.** Controlling change matters as much as adding capacity.
- **Bottlenecks move.** Fix one and the next appears, so test repeatedly.
- **Features and stability can coexist.** Feature flags separate deploying from releasing.
- **Roles must be agreed before the incident,** not during it.
- **Restore first, find the cause later.** Speed of recovery limits customer impact.
- **Readiness is a team effort.** Marketing, product and technology must share one calendar.
- **Over-provisioning costs money, outages cost more.** Decide with data, not fear.

9.3 Limitations of This Plan

- All numbers, thresholds and bottlenecks are illustrative and must be validated on the real system.
- The technology stack is unknown, so the techniques are general.
- Costs of extra capacity are not estimated.
- Fraud and security defence are covered briefly, not in depth.
- Third-party failures can be mitigated but not fully controlled.